

Product Requirements Document

A Python Reference Implementation of a Hybrid ECC–Symmetric Mutual Authentication Protocol for IoT Devices

*Proof-of-Concept Library, Demo Device/Server, and
Comparative Benchmarking Harness*

Document status:	Draft v0.1 for review
Source paper:	Al-Rasheed et al., “A Low-Cost Hybrid Elliptic Curve Cryptography Authentication Protocol for Trustworthy Internet of Things Communication,” <i>IEEE Trans. Consumer Electronics</i> , vol. 72, no. 2, pp. 4483–4491, May 2026.
Protocol basis:	Section III (nonce-based mathematical model)
Target deliverable:	Python library + CLI demo (device, server, TA)
Prepared for:	Engineering / Security review
Date:	August 25, 2026

This document defines scope, architecture, functional and non-functional requirements, milestones, and acceptance criteria for a laptop-runnable proof-of-concept. It is **not** a certification of production readiness; see Section 7 for explicit non-production caveats.

Contents

1	Purpose and Background	3
1.1	Purpose	3
1.2	Background Summary	3
1.3	Problem Statement	3
2	Goals and Non-Goals	3
2.1	Goals	4
2.2	Non-Goals	4
3	Users and Use Cases	4
4	System Overview and Architecture	5
4.1	High-Level Architecture	5
4.2	Runtime Topology (Demo)	5
4.3	Transport	6
5	Functional Requirements	6
5.1	FR-1: Cryptographic Primitives	6
5.2	FR-2: Offline Registration (TA Phase)	6
5.3	FR-3: Pairwise Symmetric Root Derivation	6
5.4	FR-4: Pseudonym Generation and Rotation	6
5.5	FR-5: Two-Message Mutual Authentication	6
5.6	FR-6: Login Phase (pre-authentication gate)	7
5.7	FR-7: Coverage / Access-Control Predicate	7
5.8	FR-8: Optional Rekeying (v2 backlog)	7
6	Benchmarking Requirements	7
6.1	Objective	7
6.2	Methodology and Disclosed Deviation	7
6.3	Communication Cost Benchmark (Table IV / Fig. 2 equivalent)	8
6.4	Processing Cost Benchmark (Table V / Fig. 1 equivalent)	8
6.5	Storage Cost Benchmark (Table VI / Fig. 3 equivalent)	8
6.6	Reporting	8
7	Security Requirements and Threat Model	9
7.1	Threat Model (per Section III-M of the paper)	9
7.2	Required Adversarial Test Cases	9
7.3	Explicit Non-Production Disclaimers	9
8	Non-Functional Requirements	9
9	API Surface (Illustrative)	10
10	Milestones and Delivery Plan	10
11	Acceptance Criteria (Definition of Done)	11
12	Risks and Open Questions	11
13	V2 / Backlog Items	12
A	Symbol-to-Code Traceability (excerpt)	12

B References

1 Purpose and Background

1.1 Purpose

This PRD defines the requirements for building a proof-of-concept (PoC) Python software package that implements the hybrid Elliptic Curve Cryptography (ECC) + symmetric-key mutual authentication protocol described in the referenced paper, along with a runnable demo (a simulated “device” process and a “server” process talking over a local network) and a benchmarking harness that reproduces the paper’s comparative evaluation (communication cost, computation/processing cost, and storage cost) against several prior schemes it cites.

The goal is to move from a paper design to working, testable, benchmarkable code that a security reviewer or engineering team can inspect, run, and extend – not to ship production IoT firmware.

1.2 Background Summary

The source paper proposes an authentication scheme for IoT networks composed of three phases:

- (a) **Offline registration** – every device/server obtains a long-term ECC-derived secret from a Trusted Authority (TA) via an ECDH-style exchange, executed once, out of band, before deployment.
- (b) **Login** – a device presents a MAC address and a hash digest of (MAC, password) to a server to establish that it is a known, registered entity.
- (c) **Mutual authentication** – device and server exchange exactly two messages to prove liveness and mutual trust and to derive a shared ephemeral session key, after which normal application traffic can flow.

The paper presents two descriptions of phase (c): a narrative algorithmic version in Section II (Algorithms 1–2) that reuses a static hash digest H_D and random value R_D across sessions, and a more rigorous mathematical model in Section III that introduces per-session nonces N_i, N_j , an epoch-rotated pseudonym $PID_{V_i,t}$, a key-derivation function (KDF), and an explicit security sketch (mutual authentication, key secrecy, unlinkability) under a Dolev–Yao-style adversary.

Design decision (confirmed with stakeholders)

The literal Section II algorithm reuses the same H_D/R_D pair across authentication attempts with no per-session nonce, which is replayable by an eavesdropper who records one valid exchange. This PoC therefore implements **Section III’s nonce-based model** (fresh N_i, N_j per session, ephemeral session key $K_{ij,t}$, rotating pseudonyms), since it is the version the paper’s own security proof sketch actually covers. Field naming is kept close to the paper’s notation ($PID, C_1, \mu_1, C_2, \sigma_2, \lambda_X, k_{ij}^*$) to keep code traceable back to the source equations (Eq. 8–23 in the paper).

1.3 Problem Statement

There is currently no working implementation of this protocol – only a paper description validated in OM-NET++ simulation. Engineering has no way to: (1) confirm the protocol is implementable as specified without hidden ambiguities, (2) verify its claimed security properties with concrete test vectors, or (3) validate the paper’s quantitative claims (approx. 17% lower communication cost, 21% lower processing cost, 12% lower storage cost versus prior schemes) against an independent, code-based measurement rather than a simulator.

2 Goals and Non-Goals

2.1 Goals

- **G1** Implement the offline registration (TA $\leftrightarrow$ device/server), pseudonym derivation, and the two-message mutual authentication exchange from Section III as a well-tested Python library.
- **G2** Provide a runnable demo: a TA provisioning CLI, a device process, and a server process, communicating over local TCP/HTTP, so a reviewer can watch a full registration + authentication flow end to end on one laptop.
- **G3** Implement an adversary test-harness that exercises the threat model in Section III-M (eavesdrop, replay, drop, inject) and demonstrates that the implementation rejects replayed/tampered messages.
- **G4** Build a benchmarking module that measures, on real code: (i) wall-clock computation cost per role, (ii) wire-format message sizes (communication cost), and (iii) persisted-state size per entity (storage cost).
- **G5** Re-implement simplified, cost-equivalent models of at least the schemes explicitly compared in Tables IV–VI (Turkanovic et al., Porambage et al., Dhillon & Karla, Challa et al., Sadhukhan et al., and 2–3 others feasible within scope) sufficient to reproduce bar charts analogous to the paper’s Figures 1–3.
- **G6** Produce a written comparison report (numbers + charts) showing where our measured results agree or disagree with the paper’s reported percentages, with an explanation of any methodological differences.
- **G7** Ship documentation sufficient for a new engineer to install, run the demo, run the test suite, and run the benchmark suite within 30 minutes.

2.2 Non-Goals

- **NG1** This is **not** an embedded/firmware implementation; no MCU, RTOS, or radio stack (XBee, LoRa, Zigbee) work is in scope.
- **NG2** This is **not** a production security product. No key-management HSM integration, no formal verification (e.g., ProVerif/Tamarin) of the protocol, no penetration-test engagement.
- **NG3** Mobility, hand-off, and coverage-radius modeling (Section III-C, Table II’s “Hand-off Time,” “device Speed”) are explicitly out of scope for v1 – the paper itself states the scheme currently targets static devices.
- **NG4** Re-implementing the OMNET++ simulation itself is out of scope; we validate via direct code execution instead (see 6.2 for why this is a deliberate, disclosed methodological deviation).
- **NG5** TA-assisted periodic rekeying (Section III-N) is marked “optional” in the source paper and is deferred to a v2 backlog item (13).

3 Users and Use Cases

User	Need	How this PoC serves it
Security reviewer / auditor	Verify the protocol’s claimed properties against a concrete artifact, not just prose	Library with unit tests per security goal (auth, key secrecy, unlinkability, replay-rejection) and a documented threat-model test suite
Research / applied-crypto engineer	Reproduce and sanity-check the paper’s performance claims	Benchmark harness producing directly comparable numbers and charts
Product engineering	Decide whether to invest in a hardened, production version	Working reference implementation + gap list between PoC and production-readiness

User	Need	How this PoC serves it
Future firmware team	Reuse validated protocol logic when porting to embedded targets	Protocol state machine and message formats documented independently of the demo transport, so core logic can be ported

4 System Overview and Architecture

4.1 High-Level Architecture

The PoC is a single Python package, `hybrid_ecc_auth`, structured into a transport-agnostic protocol core and thin demo adapters:

```
hybrid_ecc_auth/
  crypto/
    curve.py      # ECC group params, scalar mult (wraps cryptography lib)
    kdf.py        # HKDF-SHA256 wrapper (Eq. 9, 16)
    hashes.py     # H1, H2, H3 hash wrappers (Eq. 8, 11, 24)
    aead.py       # AES-256-GCM authenticated encryption (Enc/Dec, MAC)
  protocol/
    ta.py         # Trusted Authority: registration issuance logic
    entity.py     # Shared base for Device/Server (secrets, pseudonyms)
    device.py     # Device-side state machine (Msg-1 build, Msg-2 verify)
    server.py     # Server-side state machine (Msg-1 verify, Msg-2 build)
    messages.py   # Wire-format (de)serialization, versioned schema
    errors.py     # AuthenticationError, ReplayDetected, IntegrityError
  storage/
    keystore.py   # Local encrypted-at-rest secret storage (demo-grade)
    session_log.py # Nonce/pseudonym replay-window bookkeeping
  demo/
    ta_cli.py     # hea-ta CLI: offline registration tool
    device_node.py # hea-device CLI: runnable device process (TCP client)
    server_node.py # hea-server CLI: runnable server process (TCP/HTTP)
  bench/
    proposed.py   # Cost model driven by real crypto calls (this scheme)
    baselines/    # One module per compared scheme (Sec. 6.5)
    harness.py    # Runs N trials, collects timing/size/storage stats
    report.py     # Renders Table IV-VI and Figure 1-3 equivalents
  tests/
    unit/ ... property/ ... adversarial/ ...
```

4.2 Runtime Topology (Demo)

1. **TA provisioning (offline, once):** `hea-ta enroll` generates λ_X for each entity per Eq. 8, executed entirely locally (no network) to mirror the paper’s “intrusion-free offline phase.” Output: a per-entity credential bundle file.
2. **Server bring-up:** `hea-server start -port 8443` loads its credential bundle and listens for authentication requests.
3. **Device bring-up:** `hea-device authenticate -server 127.0.0.1:8443` loads its credential bundle, computes M_1 (Eq. 15), sends it, receives M_2 (Eq. 19), verifies it, and on success prints the derived session key fingerprint and round-trip latency.
4. **Adversary demo mode:** `hea-device authenticate -replay-last` deliberately resends a captured M_1 to demonstrate the server’s nonce/pseudonym freshness check rejecting it.

4.3 Transport

Local TCP sockets with length-prefixed framed JSON (binary fields base64-encoded) for the demo; the protocol core itself has no transport dependency, so a future implementer can drop in CoAP, MQTT, or a raw radio link without touching `protocol/`.

5 Functional Requirements

5.1 FR-1: Cryptographic Primitives

- **FR-1.1** Elliptic curve: NIST P-256 (secp256r1) via the `cryptography` package, satisfying the paper’s abstract $(\mathbb{F}_q, E/\mathbb{F}_q, G)$ of prime order n (Section IV-A).
- **FR-1.2** Hash function $H_{1,2,3}$: SHA-256, per the paper’s “Instantiation Notes.”
- **FR-1.3** KDF: HKDF-SHA256 (RFC 5869) for Eq. 9, Eq. 16, and Eq. 24.
- **FR-1.4** Symmetric AEAD (Enc/Dec + MAC in Eq. 13, 14, 17, 18): AES-256-GCM, using the GCM tag as the MAC per the paper’s instantiation note.
- **FR-1.5** Nonce generation: CSPRNG, $\ell = 128$ bits by default (configurable to 192/256 per Section III-K).

5.2 FR-2: Offline Registration (TA Phase)

- **FR-2.1** Given a master secret s and an entity identity ID_X , compute $\lambda_X = H_1(ID_X \| s) \bmod n$ (Eq. 8).
- **FR-2.2** Optionally compute and export the public point $P_X = \lambda_X G$.
- **FR-2.3** Persist λ_X into an entity-specific credential bundle via an explicitly offline code path (no network I/O permitted in this module – enforced by a unit test that asserts no socket is opened).
- **FR-2.4** Support batch enrollment of N devices and M servers from a manifest file (to drive Section 6 benchmarking at the paper’s tested scale of 100–1000 devices, Table II).

5.3 FR-3: Pairwise Symmetric Root Derivation

- **FR-3.1** Derive $k_{ij}^* = \text{KDF}(\lambda_{V_i} \| \lambda_{R_j})$ per Eq. 9 on both sides independently (device knows λ_{R_j} from its TA-issued peer-key list; server knows λ_{V_i} analogously), reproducing the paper’s “list of secret keys” distribution model from Section II.
- **FR-3.2** Cache derived k_{ij}^* values to avoid recomputation cost on every session (with an explicit cache-invalidation hook for FR-8, TA-assisted rekey).

5.4 FR-4: Pseudonym Generation and Rotation

- **FR-4.1** Compute $\text{PID}_{V_i,t} = H_2(\lambda_{V_i} \| t)$ (Eq. 11) where t is an epoch counter, configurable epoch length (default: 1 authentication session).
- **FR-4.2** Server resolves an incoming pseudonym to a known device identity via a local lookup table it maintains as part of registration state (constant-time compare to avoid trivial timing side-channels).

5.5 FR-5: Two-Message Mutual Authentication

- **FR-5.1 Message 1 (Device → Server):** device samples nonce N_i , computes $C_1 = \text{Enc}_{k_{ij}^*}(\text{PID}_{V_i,t} \| N_i \| t)$ and $\mu_1 = \text{MAC}_{k_{ij}^*}(\text{PID}_{V_i,t} \| N_i \| t)$, sends $M_1 = (\text{PID}_{V_i,t}, C_1, \mu_1)$ (Eq. 12–15).
- **FR-5.2 Server processing:** resolve pseudonym → recompute $k_{ij}^* \rightarrow$ verify $\mu_1 \rightarrow$ decrypt $C_1 \rightarrow$ reject on any failure with a typed `IntegrityError / ReplayDetected` exception; on success sample N_j and derive $K_{ij,t} = \text{KDF}(k_{ij}^* \| N_i \| N_j \| t)$ (Eq. 16).

- **FR-5.3 Message 2 (Server → Device):** server computes $\sigma_2 = \text{MAC}_{k_{ij}^*}(\text{PID}_{V_i,t} \| N_i \| N_j \| t)$, $C_2 = \text{Enc}_{k_{ij}^*}(N_j \| t)$, sends $M_2 = (\text{ID}_{R_j} \text{ or } \text{PID}_{R_j,t}, C_2, \sigma_2)$ (Eq. 17–19).
- **FR-5.4 Device verification:** verify σ_2 , decrypt C_2 , confirm returned N_j matches server’s claim, derive the same $K_{ij,t}$, and assert $K_{ij,t}^{(V)} = K_{ij,t}^{(R)}$ (correctness property, Eq. 20) – enforced as an integration-test invariant, not just documentation.
- **FR-5.5** Enforce a configurable per-session timeout window (default 5s, tunable) after which an incomplete handshake is aborted and counted as a failed attempt.
- **FR-5.6** Enforce the paper’s “maximum three attempts” rule (Section II): after 3 consecutive failed authentication attempts from the same MAC/identity, the entity is added to a local block-list and subsequent attempts are rejected without cryptographic processing (fail fast), and (per Algorithm 1/2, lines 14–15) the blocked identifier is broadcast to other locally-known trusted peers via a simple pub/sub demo channel.

5.6 FR-6: Login Phase (pre-authentication gate)

- **FR-6.1** Implement the MAC-address + $H_D = h(\text{MAC}_D \| \text{PWD})$ login gate described in Section II-B as an optional, togglable pre-check before the Section III authentication exchange, since the source paper treats login and mutual authentication as sequential phases.

5.7 FR-7: Coverage / Access-Control Predicate

- **FR-7.1** Implement $\text{cov}_{ij}(t)$ (Eq. 10) as a pluggable predicate; for the laptop PoC this defaults to `True` (always in range), with a configuration flag to simulate out-of-range rejection for demo/testing purposes (Eq. 23, $\text{Allow}_{ij}(t)$).

5.8 FR-8: Optional Rekeying (v2 backlog)

- **FR-8.1** Provide a stub interface for Eq. 24 periodic refresh, unimplemented in v1, to avoid architectural rework later.

6 Benchmarking Requirements

6.1 Objective

Reproduce, as closely as methodologically honest, the three comparative evaluations in the paper (Tables IV, V, VI and Figures 1–3): communication overhead (bits transmitted), processing/computation overhead (time), and storage overhead (bytes retained), for our implementation versus re-implemented baseline cost models of competing schemes.

6.2 Methodology and Disclosed Deviation

The paper’s numbers come from an OMNET++ network simulation with the parameters in Table II (100–1000 devices, 10 Mbps channel, specific energy-per-bit constants, etc.) and fixed per-operation timing constants in Table III ($T_{ECM} = 17.1\text{ms}$, $T_H = 0.32\text{ms}$, $T_{SE/D} = 5.6\text{ms}$, etc.) that were themselves *assumed*, not measured on real hardware in that paper.

This PoC will **measure real operation costs on the benchmarking machine** (actual scalar multiplication time, actual AES-GCM time, actual hash time using Python’s `cryptography` bindings to OpenSSL) rather than reusing the paper’s assumed constants, because:

1. It gives an independently verifiable number instead of repeating an unverified assumption.

2. Table III’s constants were plainly calibrated for resource-constrained embedded hardware (note $T_{ECM} = T_{fe} = 17.1\text{ms}$ – values typical of 8/16-bit microcontrollers), which a laptop-based PoC cannot faithfully emulate.

The final report must explicitly state both: (a) our laptop-measured absolute numbers, and (b) a re-derived relative-comparison table using the paper’s own Table III constants plugged into each scheme’s published operation-count formula (Table V’s column expressions, e.g. $7T_H, 1T_{fe} + 13T_H + 2T_{SE/D}$), so readers can see the claims reproduced analytically even where our absolute wall-clock numbers diverge due to hardware differences.

6.3 Communication Cost Benchmark (Table IV / Fig. 2 equivalent)

- **BR-1.1** Serialize real M_1/M_2 wire messages and measure their exact byte length (not an assumed 128-bit/message constant), broken down by field (PID, ciphertext, tag) so the report can show where bytes go.
- **BR-1.2** Compute the offline (registration) message sizes separately from the per-session (authentication) sizes, and report both, since the paper conflates “4-message” totals across different phases for baseline schemes.
- **BR-1.3** For baseline schemes (Turkanovic et al., Porambage et al., Dhillon & Karla, Cheng & Le, Jiang et al., Shuai et al., Wazid et al., Sadhukhan et al., Fakroon et al.), implement a cost-model stub that reproduces their reported per-entity bit counts from Table IV as a static reference (source: paper Table IV), clearly labeled as “reference value from source paper,” not independently re-derived from re-implemented baseline protocols (full re-implementation of nine external protocols is out of scope for v1 – see 13).

6.4 Processing Cost Benchmark (Table V / Fig. 1 equivalent)

- **BR-2.1** Time our real device-side and server-side cryptographic operations (KDF calls, AEAD encrypt/decrypt, MAC verify) over ≥ 1000 trials, reporting mean, median, p95, and stddev, on the benchmarking host.
- **BR-2.2** Additionally compute the same operation-count formulas from Table V symbolically (e.g., our scheme: $2T_H + 2T_{SE/D}$ client-side, $4T_H + 5T_{ECM}$ server-side per the paper) using *both* (a) our measured per-primitive timings and (b) the paper’s Table III constants, to produce a side-by-side of “paper-assumed” vs. “our-measured” processing cost.
- **BR-2.3** Reproduce a Figure-1-style bar chart (processing cost vs. number of devices, mirroring the paper’s “vehicles” axis relabeled “devices”) using matplotlib, saved as a reproducible artifact (PNG + underlying CSV).

6.5 Storage Cost Benchmark (Table VI / Fig. 3 equivalent)

- **BR-3.1** Measure actual serialized byte size of the per-entity persisted state (credential bundle: λ_X , cached k_{ij}^* list, block-list, pseudonym epoch counter) for both device and server roles.
- **BR-3.2** Compare against the paper’s reported Table VI values (Proposed: 320 bytes server / 320 bytes device) as static reference baselines for the other schemes, flagged as source-paper values.

6.6 Reporting

- **BR-4.1** `bench/report.py` generates a self-contained Markdown + HTML report with all three chart types, a methodology-deviation callout box (per 6.2), and a machine-readable JSON of all raw numbers for reproducibility/CI regression tracking.
- **BR-4.2** CI runs a lightweight version of the benchmark (reduced trial count) on every merge to `main` to catch performance regressions; the full benchmark (paper-scale, 100–1000 simulated devices) runs on demand / nightly.

7 Security Requirements and Threat Model

7.1 Threat Model (per Section III-M of the paper)

The adversary $\mathcal{A}$ is assumed to be a probabilistic polynomial-time (PPT) attacker who fully controls the communication channel: can eavesdrop, replay, drop, reorder, and inject messages, but **cannot** compromise the offline TA registration phase and cannot extract λ_X from a device without physical/implementation-level compromise (side-channel attacks against the local key store are explicitly out of scope for this PoC).

7.2 Required Adversarial Test Cases

Test ID	Scenario	Expected Result
SEC-01	Replay a previously captured, valid M_1	Rejected: stale nonce/pseudonym epoch detected
SEC-02	Replay a previously captured, valid M_2 to a different device session	Rejected: σ_2 /nonce binding mismatch
SEC-03	Flip a single bit in C_1 or μ_1 in transit	Rejected: AEAD tag / MAC verification failure
SEC-04	Attacker without λ_{V_i} attempts to forge M_1 for a known PID	Rejected: cannot produce valid μ_1
SEC-05	3 consecutive failed attempts from same identity	Entity is block-listed; 4th attempt short-circuited pre-crypto
SEC-06	Two devices' pseudonyms compared across epochs by a passive observer	Statistical indistinguishability check (unlinkability smoke test, not a formal proof)
SEC-07	Session key comparison across two independent honest runs	Keys differ (freshness / no key reuse across sessions)
SEC-08	Out-of-range device (coverage predicate = False) attempts authentication	Rejected at $\text{Allow}_{ij}(t)$ gate regardless of crypto validity

7.3 Explicit Non-Production Disclaimers

- This PoC's local key storage (`storage/keystore.py`) is file-based with OS-level file permissions only; it is **not** equivalent to a hardware security module or secure element and must not be used to store real secrets.
- No formal/mechanized protocol verification (Tamarin, ProVerif, or similar) is performed in v1; the security test suite in 7 is a set of targeted adversarial unit tests, not a proof.
- Timing side-channels, power-analysis side-channels, and fault-injection attacks are out of scope.
- The paper does not fully specify collision/downgrade handling when t (epoch) clocks drift between device and server; this PoC will document its own chosen tolerance window as an implementation decision, flagged for security review.

8 Non-Functional Requirements

Category	Requirement
Correctness	100% of protocol state-machine branches covered by unit tests; correctness invariant (Eq. 20) verified in $\geq 10,000$ randomized trials with zero failures
Performance	Full two-message authentication (excluding network latency) completes in $< 5\text{ms}$ median on a modern laptop CPU
Portability	Pure Python ≥ 3.11 , cross-platform (Linux/macOS/Windows), single <code>pip install -e . setup</code> , no native build step beyond the cryptography wheel
Reproducibility	Benchmarks pinned to a <code>requirements-lock.txt</code> ; report includes CPU model, Python version, and library versions used
Test coverage	$\geq 90\%$ line coverage on <code>protocol/</code> and <code>crypto/</code> packages, enforced in CI
Documentation	README quick-start, architecture doc, protocol-to-code equation traceability table (Appendix A), and a CHANGELOG
Logging & observability	Structured logs (JSON) for every authentication attempt (outcome, latency, entity id) without ever logging raw secret material
Licensing	Library released under a permissive license (e.g., Apache-2.0); citation back to the source paper included in README and NOTICE file

9 API Surface (Illustrative)

```
# Trusted Authority (offline)
ta = TrustedAuthority(master_secret=..., curve="secp256r1")
device_cred = ta.enroll(identity="device-001", role="device")
server_cred = ta.enroll(identity="server-A", role="server")
ta.share_peer_keys(device_cred, [server_cred]) # Sec. II peer-key list model

# Device side
device = Device(credential=device_cred, coverage_predicate=always_true)
msg1 = device.build_auth_request(peer_id="server-A") # Eq. 12-15
# ... send msg1 over transport ...
session_key = device.complete_auth(msg2_bytes) # Eq. 17-20 verify

# Server side
server = Server(credential=server_cred)
result = server.handle_auth_request(msg1_bytes) # Eq. 16, may raise
msg2 = result.response_bytes # Eq. 17-19
session_key = result.session_key
```

10 Milestones and Delivery Plan

Phase	Name	Exit Criteria
0	Crypto core	<code>crypto/</code> module complete with unit tests for curve ops, HKDF, AES-GCM wrappers; all FR-1 items done
1	TA & registration	FR-2, FR-3 implemented; <code>hea-ta enroll</code> CLI functional; offline-only invariant test passing
2	Pseudonyms & login gate	FR-4, FR-6 implemented and unit tested

Phase	Name	Exit Criteria
3	Mutual authentication core	FR-5 fully implemented; correctness invariant (Eq. 20) test suite green; local-socket demo (device ↔ server) working end to end
4	Security hardening & adversarial tests	All SEC-01..SEC-08 test cases implemented and passing; block-list (FR-5.6) functional
5	Benchmarking harness – proposed scheme	BR-1.1, BR-2.1, BR-3.1 implemented against our own real implementation
6	Benchmarking harness – baselines & report	BR-1.3, BR-2.2, BR-3.2, BR-4.1 done; full comparative report (Tables/Figures analogues) generated
7	Docs, packaging, CI, release	README, architecture doc, traceability appendix complete; CI green; tagged v0.1.0 release

11 Acceptance Criteria (Definition of Done)

1. A reviewer can run `pip install -e .`, then a documented three-terminal quick-start (TA enroll → start server → run device), and observe a successful mutual authentication with a printed session-key fingerprint, in under 5 minutes.
2. Running the replay-demo flag produces a visible, correctly classified rejection (not a crash, not a silent success).
3. `pytest` passes with $\geq 90\%$ coverage on `protocol/` and `crypto/`.
4. All eight SEC-0x adversarial tests pass.
5. `python -m hybrid_ecc_auth.bench.harness -full` produces a report containing: our measured communication/processing/storage numbers, the paper’s reference numbers for baseline schemes, and an explicit methodology-deviation note per 6.2.
6. A written summary states, per metric (communication/processing/storage), whether our reproduction is directionally consistent with the paper’s claimed 17%/21%/12% reductions, and explains any divergence.

12 Risks and Open Questions

Risk / Open Question	Notes / Mitigation
Section II and Section III of the source paper are not fully consistent (static vs. nonce-based fields)	Resolved by design decision to follow Section III exclusively; documented explicitly in README as a deviation from the literal Algorithm 1/2 pseudocode
Paper’s Table III timing constants look calibrated for embedded MCUs, not laptops	Addressed via dual-reporting methodology (6.2); do not present laptop absolute numbers as validating embedded-scale claims
Full re-implementation of 9 baseline schemes is a large, separate effort	v1 uses paper-reported reference values for baselines’ own reported metrics, clearly labeled; full baseline re-implementation is a v2 backlog candidate
Epoch/clock-drift handling for pseudonym freshness is unspecified in the paper	Implementation will define and document its own tolerance window; flagged for security review sign-off
No public test vectors exist for this protocol	We will publish our own generated test vectors in the repo so future ports (e.g., embedded) can cross-check against this reference implementation

13 V2 / Backlog Items

- Full independent re-implementation of the nine baseline schemes (rather than citing their paper-reported metrics) for a fully apples-to-apples benchmark.
- Implement optional TA-assisted rekeying (Eq. 24, Section III-N).
- Mobility / coverage-radius dynamic simulation (Section III-C) with moving device positions.
- Port protocol core to an embedded target (e.g., ESP32/ARM Cortex-M) once the PoC is validated, reusing protocol/ unchanged and swapping crypto/ bindings for a hardware-accelerated ECC library.
- Formal verification pass (Tamarin/ProVerif model) of the Section III message flow.

A Symbol-to-Code Traceability (excerpt)

Paper symbol	Eq.	Code reference
λ_X	8	ta.py :: derive_long_term_secret
k_{ij}^*	9	entity.py :: derive_pairwise_root
$\text{cov}_{ij}(t)$	10	entity.py :: coverage_predicate
$\text{PID}_{V_i,t}$	11	device.py :: current_pseudonym
N_i, C_1, μ_1, M_1	12–15	device.py :: build_auth_request
$K_{ij,t}$	16	server.py :: derive_session_key
σ_2, C_2, M_2	17–19	server.py :: build_auth_response
Correctness check	20	tests/property/ test_session_key_agreement.py
$\text{Allow}_{ij}(t)$	23	entity.py :: access_control_gate
Rekey (v2)	24	protocol/rekey.py (stub only in v1)

B References

1. Al-Rasheed, A., Khan, R., Ahmad, A., Saeed, A., Alturise, F., & Alkhalaf, S. “A Low-Cost Hybrid Elliptic Curve Cryptography Authentication Protocol for Trustworthy Internet of Things Communication.” *IEEE Transactions on Consumer Electronics*, vol. 72, no. 2, pp. 4483–4491, May 2026.
2. RFC 5869 – HMAC-based Extract-and-Expand Key Derivation Function (HKDF).
3. NIST SP 800-38D – Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM).
4. Python cryptography library documentation, <https://cryptography.io>.